

STUDY-ARCH-Scaling.md

Il Manuale Completo per Consulente di Tech Scaling & Architettura

Track: ARCH — Tech Scaling & Architecture Advisory

Autore: Elios Scoglio — per uso personale, studio e insegnamento

Versione: 1.0 — Maggio 2026

Tono: Da nerd serio che però sa quando smettere di parlare di Kubernetes.

"L'architettura è l'insieme delle decisioni che vorresti non dover prendere di nuovo." — Ralph Johnson (parafrasato da chiunque abbia sofferto un monolite a 3AM)*

Come usare questo manuale

Questo manuale copre il track ARCH: come aiutare le aziende tech in crescita a scalare la loro architettura senza implodere. Ogni parte ha teoria, esempi con personaggi inventati, alternative, trappole, e domande di verifica. Leggi tutto o saltella: è fatto per resistere a entrambe le strategie.

PARTE 1 — Cosa significa "Tech Scaling" e quando serve davvero

Il concetto

"Scaling" non è solo "reggere più traffico". È la capacità di un sistema (e del team che lo gestisce) di crescere in dimensione, complessità e velocità senza collassare.

Ci sono 3 tipi di scaling che un'azienda tech affronta:

1. Scaling tecnico — il sistema regge il carico crescente
 - Vertical scaling: macchine più potenti
 - Horizontal scaling: più istanze dello stesso componente
 - Database scaling: sharding, read replicas, CQRS
 - Caching: Redis, CDN, layer applicativi
2. Scaling architetturale — il codice non diventa un groviglio
 - Decomposizione del monolite
 - Definizione di bounded context
 - API contracts e versioning
 - Separazione deploy da release
3. Scaling organizzativo — il team non diventa un collo di bottiglia
 - Team topology (stream-aligned, enabling, platform, complicated-subsystem)
 - Ownership dei componenti
 - Cognitive load management

- Processi decisionali che scalano

Perché conta

La maggior parte delle aziende tech incontra i problemi di scaling quando è troppo tardi: il sistema già crolla sotto il peso, i deployment richiedono 2 giorni, e ogni modifica rompe qualcosa di imprevedibile. Il tuo valore è arrivare prima.

Esempio

ByteFlow, start-up SaaS da 15 sviluppatori, 50.000 utenti, crescita 20% mese/mese

Il CTO Matteo ha costruito un monolite Rails in 18 mesi. Funzionava benissimo fino a 3 mesi fa. Ora: deployment ogni 2 settimane perché "troppo rischioso farlo più spesso", 3 hotfix/settimana, 6 sviluppatori su 15 lavorano solo su bug. Il team cresce ma la velocità cala. È il pattern classico del "monolite che non scala più". Il primo step non è "fare microservizi" — è capire dove sono i confini reali del dominio e quali sono i 2-3 punti di maggiore friction.

Alternativa

Non ogni problema di scaling si risolve con architettura. Spesso il problema principale è il processo: nessun code review, nessun test, deploy manuali, nessuna osservabilità. Prima di riprogettare l'architettura, valuta se il problema è il codice o il processo che lo produce.

Cosa NON fare

Non proporre "migrare a microservizi" come soluzione generica. I microservizi non sono la risposta: sono una scelta con costi enormi (complessità operativa, latenza di rete, testing distribuito, distributed tracing). Proponi solo quando hai evidenza che il confine di dominio esiste e che il team è pronto a gestire la complessità aggiuntiva.

****Attenzione!**** *"Distributed Monolith" è il peggior dei due mondi: hai la complessità dei microservizi senza i benefici dell'indipendenza. Si riconosce quando: ogni deploy richiede di aggiornare 5 servizi in ordine preciso, o quando un servizio conosce lo schema del database di un altro. Identifica sempre questo pattern prima di proporre ulteriore decomposizione.*

****Tip da campo**** *La prima domanda in ogni assessment di scaling non è "com'è fatto il sistema?" ma "qual è il ritmo di deployment attuale e qual è il ritmo desiderato?". Se deployano 1 volta/mese e vogliono arrivare a ogni giorno, hai il problema principale su un piatto d'argento.*

Domande di verifica:

- Un'azienda dice "dobbiamo scalare". Come distingui se il problema è tecnico, architetturale, o organizzativo?
- Quali sono i segnali che indicano un "distributed monolith"?
- Perché aumentare il numero di sviluppatori spesso non risolve i problemi di scaling?

PARTE 2 — L'Architecture Assessment: capire prima di intervenire

Il concetto

Prima di raccomandare qualsiasi cambiamento architetturale, devi capire il sistema esistente. L'Architecture Assessment è il tuo strumento: un'analisi strutturata che produce una fotografia dello stato attuale e un insieme di priorità di intervento.

Struttura Architecture Assessment (2-3 settimane per sistemi medi):

Dimensione 1 — Struttura del codice

- Quanti moduli/servizi/componenti
- Tipo di accoppiamento (tight vs loose)
- Presenza di boundary espliciti
- Qualità dei contratti API (interni ed esterni)

Dimensione 2 — Deployment e operatività

- Frequenza e affidabilità dei deploy
- Tempi di build e test
- Processi di rollback
- Osservabilità (log, metriche, trace)

Dimensione 3 — Qualità del codice

- Test coverage e qualità dei test
- Complessità ciclomatica dei componenti critici
- Presenza di "hotspot" (file cambiati spesso, sempre gli stessi)
- Debito tecnico documentato vs latente

Dimensione 4 — Organizzazione e processo

- Team topology e ownership
- Cognitive load per team
- Bottleneck nelle review e negli approvals
- Cultura post-mortem e learning

Output dell'assessment:

- Architecture Map (C4 model, livello 1-2)
- Hotspot Matrix (quali componenti causano più problemi)
- Risk Register architetturale
- Roadmap prioritizzata con quick wins e interventi strategici

Perché conta

Intervenire su un sistema senza capirlo è come fare chirurgia senza radiografie. L'assessment non è burocrazia: è protezione tua e del cliente da decisioni basate su assunzioni sbagliate.

Esempio

Assessment di Cloudia (SaaS B2B, 8 sviluppatori, 3 anni di vita)

Durante l'assessment hai trovato: 1 monolite .NET con 340 controller in un unico progetto, 0% test coverage, deployment via FTP manuale su un singolo server, nessun monitoring. Hotspot: il modulo "billing" viene modificato ogni settimana ed è coinvolto in ogni bug. La priorità emersa non è "fare microservizi": è 1) aggiungere test al billing, 2) automatizzare il deploy, 3) estrarre il billing come modulo indipendente (non microservizio, modulo). Tre step in 3 mesi, prima di pensare a qualsiasi decomposizione più profonda.

Alternativa

Per aziende molto piccole (2-4 sviluppatori), un assessment completo è overkill. Proponi una Architecture Conversation: 3 ore con il team, una serie di domande guidate, e un documento di 2 pagine con le 5 priorità. Prezzo: 500-800€.

Cosa NON fare

Non fare l'assessment da solo senza coinvolgere il team. Le persone che vivono il sistema ogni giorno hanno informazioni che non emergono da nessun documento. E se non le coinvolgi, non daranno mai credito alle tue raccomandazioni.

*****Attenzione!**** I "hotspot" non sono sempre dove il team pensa che siano. Fai sempre un'analisi git: ``git log --format=format: --name-only | sort | uniq -c | sort -rg | head -20``. I file con più commit sono quasi sempre i bottleneck reali. Questo numero non mente.*

*****Tip da campo**** Quando presenti i risultati dell'assessment, non presentare una lista di problemi: presenta una narrazione. "Il vostro sistema ha tre anni di storia costruttiva, e questo si vede. Il 'billing' è il cuore del vostro business e si è mangiato il 40% delle modifiche degli ultimi 12 mesi. La buona notizia è che sappiamo esattamente dove intervenire." Questo vende la roadmap molto meglio di un elenco di debiti tecnici.*

Domande di verifica:

- Come strutturaresti un'Architecture Assessment per un sistema di cui non hai nessuna documentazione?
- Quali strumenti usi per fare un'analisi di hotspot su un codebase git?
- Come presenti i risultati di un assessment che rivela problemi molto gravi senza demoralizzare il team?

PARTE 3 — Monolite Modulare vs Microservizi: il dibattito finisce qui

Il concetto

La scelta non è binaria. Il continuum architetturale è:

Monolite spaghetti → Monolite modulare → Macrosvizi → Microservizi → Nanoservizi (perché no, qualcuno ci prova)

Monolite modulare: un unico processo deployabile, ma con boundary espliciti e ben definiti all'interno del codice. Moduli con interfacce pubbliche chiare, nessuna dipendenza trasversale non autorizzata, database logicamente separati (anche se fisicamente unico).

Quando il monolite modulare è la scelta giusta:

- Team < 10 sviluppatori
- Dominio non ancora stabilizzato (i confini cambiano spesso)
- Latency tra componenti è critica
- Maturità operativa bassa (nessuno sa fare K8s/service mesh)
- Vuoi tutti i benefici del refactoring senza il costo operativo dei microservizi

Microservizi: processi indipendenti, deployabili separatamente, con ownership di dati propria, comunicanti via API/eventi.

Quando i microservizi sono la scelta giusta:

- Team > 10-15 sviluppatori che devono lavorare in parallelo
- Parti del sistema con requisiti di scaling molto diversi
- Necessità di deploy indipendente per velocità di iterazione
- Boundary di dominio ben stabilizzati (non cambieranno a breve)
- Maturità operativa alta (sai fare CI/CD, K8s, tracing distribuito, chaos engineering)

Perché conta

La decisione sbagliata costa anni. Fare microservizi troppo presto = distributed monolith + overhead operativo enorme + team frustrato. Restare nel monolite troppo a lungo = bottleneck di deploy, scaling impossibile, cognitive load ingestibile.

Esempio

Il caso di Nexio (e-commerce B2B, 200K clienti, 20 sviluppatori)

Nexio ha un monolite PHP da 5 anni, 600K righe di codice. Il team vuole "fare microservizi come Netflix". Tu fai l'assessment e trovi: 3 parti del sistema hanno frequenze di deploy molto diverse (catalog: 1/settimana; payments: 3/giorno; user management: 1/mese), e 2 parti hanno requisiti di scaling diversi (search: spike enormi; checkout: costante). Raccomandazione: non "fare microservizi", ma estrarre 2 bounded context ad alta criticità (payments, search) come macroservizi, lasciando il resto nel monolite. Questo riduce il rischio del 80% e porta l'80% del beneficio.

Alternativa

Strangler Fig Pattern: non migrare il sistema tutto insieme. Costruisci il nuovo sistema intorno al vecchio, reindirizzando gradualmente il traffico. Il vecchio "muore" per strangolamento (senza violenza, è solo una pianta). Questo permette di validare ogni pezzo del nuovo design prima di buttare il vecchio.

Cosa NON fare

Non fare la scelta architetturale sulla base di cosa "suona meglio" in conferenza. "Microservizi" è sexy nel 2024, ma se il tuo cliente ha 4 sviluppatori e un database Oracle legacy, i microservizi sono un percorso verso il disastro.

****Attenzione!**** Il "confine del dominio" non lo decide il CTO guardando la whiteboard. Emerge dall'analisi del linguaggio del business (Ubiquitous Language nel DDD), dai flussi di dati, e da dove i team naturalmente si separano. Se i confini non emergono dalla realtà, qualsiasi architettura costruita su di essi sarà artificiale e fragile.

****Tip da campo**** Quando valuti se un pezzo di monolite è "pronto" per essere estratto, fai il test delle "3D": Dati separati (ha il suo schema?), Deploy indipendente (si può deployare senza toccare il resto?), Domain owner (c'è un team che lo possiede?). Se risponde sì a tutte e 3: candidato credibile. Se risponde no a una o più: aspetta.

Domande di verifica:

- Un cliente con 8 sviluppatori dice "voglio passare a microservizi perché così scaliamo meglio". Come valuti la richiesta?
- Cosa è il "Strangler Fig Pattern" e quando lo proponi?
- Quali sono le 3 precondizioni organizzative (non tecniche) per fare microservizi con successo?

PARTE 4 — API Design e Versioning: il patto con il futuro

Il concetto

Un'API pubblica è un contratto. Una volta rilasciata e adottata da altri, cambiarla ha costi enormi. Progettare API bene fin dall'inizio non è perfezionismo: è rispettare il tempo degli altri.

Principi fondamentali di API design:

1. API-First: progetta l'interfaccia prima di scrivere il codice. OpenAPI 3.1 come spec, la spec guida l'implementazione, non il contrario.
2. Naming coerente: risorse come sostantivi plurali (/orders, non /getOrders). Nessun verbo nei path. HTTP methods per le azioni (GET, POST, PUT, PATCH, DELETE).
3. Versioning semantico: major version nel path (/v1/orders). Breaking change = nuovo major. Minor change (aggiunta campo opzionale) = no versioning change.
4. Error model standardizzato: RFC 7807 (application/problem+json). Non inventare il tuo formato errori.
5. Idempotenza dove necessario: operazioni critiche (pagamento, emissione, cancellazione) devono essere idempotenti.

Usa Idempotency-Key header.

6. Paginazione server-driven: non caricare 100K record su client. Cursor-based pagination per dataset grandi.

7. Documentazione come prodotto: la doc non è un allegato. È il prodotto. Se la doc è sbagliata, l'API è sbagliata.

Perché conta

Un'API mal progettata sopravvive per anni. Ho visto API con nomi come `/doUpdateUserAndSendEmail` che esistevano da 2013 in produzione e nessuno sapeva cambiarle perché 40 client le usavano. Il costo del refactoring API è 10x il costo di progettarle bene la prima volta.

Esempio

L'API di TicketZen (immaginaria) per la prenotazione posti

```
// Cattivo:
POST /bookSeat?userId=123&eventId=456&seatId=789

// Buono:
POST /v1/events/{eventId}/reservations
{
  "seatId": "789",
  "customerId": "123"
}

// Response:
201 Created
{
  "reservationId": "res_xyz",
  "status": "confirmed",
  "expiresAt": "2026-05-27T15:30:00Z"
}
```

Il "Cattivo" non è restful, espone logica nel path, non versiona, e non ha idempotenza. Il "Buono" è resource-oriented, versionato, restituisce lo stato completo della risorsa creata.

Alternativa

Per API interne tra microservizi, gRPC + Protobuf è spesso migliore di REST: schema-first, più efficiente in rete, generazione automatica di client e server stubs, streaming nativo. Il trade-off è la leggibilità (non si può testare con curl facilmente) e il tooling più complesso.

Cosa NON fare

Non fare `PUT /api/v1/user/update`. Non fare `/getAll`. Non fare errori con HTTP 200 e body `{"success": false}`. E soprattutto non fare versioning con `?version=2` in query string: è un incubo per il caching.

****Attenzione!**** Il più grande errore nell'API design è l'**under-versioning**: modificare una API senza aumentare la versione perché "è solo un campo aggiunto". Anche un campo opzionale aggiunto può rompere un client se il client valida strettamente lo schema (e molti lo fanno). Regola d'oro: se non sei sicuro se è breaking, trattala come breaking.

****Tip da campo**** Prima di pubblicare un'API esternamente, fai una "API Review" con almeno un consumer potenziale. Passa i path, i payload, e gli scenari di errore. Il feedback in questa fase è gratuito: dopo che è in produzione costa carissimo.

Domande di verifica:

- Come struttureresti il versioning di un'API che ha bisogno di deprecare un campo in un response?
- Qual è la differenza tra PUT e PATCH? Quando usi l'uno e quando l'altro?
- Come implementeresti l'idempotenza per un endpoint di pagamento?

PARTE 5 — Database Patterns per sistemi che crescono

Il concetto

Il database è spesso il primo collo di bottiglia quando un sistema scala. I pattern per gestirlo:

1. Read Replicas: il master gestisce le write, le repliche gestiscono le read. Ottimo per read-heavy workload. Attenzione alla replication lag: la replica potrebbe non essere aggiornata.
2. CQRS (Command Query Responsibility Segregation): separa il modello di scrittura (comandi) dal modello di lettura (query). Le query usano un modello ottimizzato per la lettura (denormalizzato, eventualmente proiettato su uno storage diverso). Complessità alta: usa solo quando hai un vero bisogno.
3. Event Sourcing: non salvare lo "stato corrente" ma la sequenza di eventi che hanno portato a quello stato. Ogni "fatto" è immutabile. Lo stato corrente si ricostruisce riproducendo gli eventi. Ottimo per audit trail, time travel, retrocompatibilità. Complessità altissima.
4. Sharding: dividere i dati su più database in base a una chiave (shard key). Scala orizzontalmente le write. Complessità enorme: query cross-shard costose, re-sharding doloroso. Usa solo se hai davvero esaurito le altre opzioni.
5. Caching con invalidation strategy: Redis/Memcached per tenere in memoria i dati più acceduti. La difficoltà è l'invalidazione: quando il cache diventa stale? Write-through, write-around, write-back: conosci le differenze e scegli in base al workload.
6. Database per dominio: ogni bounded context ha il suo schema (o database separato). Nessun join cross-dominio. La consistenza eventuale è il prezzo da pagare.

Perché conta

Il 60% dei problemi di performance in produzione che ho visto erano problemi di database: query N+1, indici mancanti, transazioni troppo lunghe, connessioni non rilasciate. Prima di qualsiasi redesign architetturale, fai sempre un database health check.

Esempio

Il N+1 Problem di CatalogHub

CatalogHub ha un catalogo prodotti con 50.000 items. Ogni item ha una lista di attributi (colori, taglie, materiali). Il codice:

```
// Il codice "ovvio" che uccide il database:
var items = db.Items.ToList(); // 1 query
foreach (var item in items)
{
    var attributes = db.Attributes.Where(a => a.ItemId == item.Id).ToList(); // N query
}
// Totale: 50.001 query al database. In produzione con 100 utenti: 5M query al secondo.

// La soluzione:
var items = db.Items.Include(i => i.Attributes).ToList(); // 1 query con JOIN
// oppure
var items = db.Items.Select(i => new ItemDto { ... i.Attributes ... }).ToList(); // 1 query con proiezione
```

Questo bug era in produzione da 2 anni. Scoperto con un profiler in 15 minuti. Fix in 5 minuti. Risultato: query time da 8 secondi a 200ms.

Alternativa

Non serve sempre un RDBMS. Valuta:

- Redis per session data, rate limiting, real-time counters
- Elasticsearch per full-text search su dati strutturati
- MongoDB per documenti senza schema rigido (ma attento ai join: li farai mancare)
- ClickHouse per analytics su log e eventi ad alta velocità
- TimescaleDB per serie temporali (metriche, IoT)

Cosa NON fare

Non usare il database come message queue. Non fare polling su tabelle per simulare eventi. Non tenere connessioni aperte per tutta la durata di una richiesta HTTP. E mai, mai, "SELECT *" in produzione.

****Attenzione!**** La migrazione di schema su un database in produzione con milioni di righe è un'operazione ad alto rischio. Un `ALTER TABLE ADD COLUMN NOT NULL` senza default può bloccare la tabella per ore. Usa sempre: migrazioni in forward-only, zero-downtime migration pattern (add column nullable, backfill, add constraint, drop old column in step separati).

****Tip da campo**** Il query plan è il tuo migliore amico. `EXPLAIN ANALYZE` in Postgres, `EXPLAIN` in MySQL, il Query Store in SQL Server. Prima di ottimizzare, guarda il piano. Il database ti dice esattamente dove perde tempo, se glielo chiedi.

Domande di verifica:

- Come spiegheresti la differenza tra CQRS e Event Sourcing a un team che li confonde?
- Quando useresti un'architettura con database separati per bounded context, e quali problemi crea?
- Quali sono le 3 migrazioni di schema più pericolose in produzione?

PARTE 6 — Resilienza e Circuit Breaker

Il concetto

In un sistema distribuito, i fallimenti sono normali. Non "possono succedere": succedono. L'architettura resiliente non previene i fallimenti, li contiene.

Pattern di resilienza fondamentali:

Timeout: ogni chiamata remota deve avere un timeout. Senza timeout, una chiamata bloccata può saturare il pool di thread e buttare giù l'intero sistema.

Retry con backoff esponenziale: riprova dopo un errore transiente, ma aspetta sempre di più tra un tentativo e l'altro. Aggiungi jitter (randomizzazione) per evitare thundering herd (tutti i client che riprovano nello stesso momento).

Circuit Breaker: dopo N fallimenti consecutivi, il circuito "apre" e smette di fare chiamate al servizio degradato (fail-fast). Dopo un periodo, "si richiude" e riprova. Protegge il sistema da cascate di fallimenti.

Bulkhead: isola le risorse critiche. Se un servizio B è lento, non deve poter esaurire il pool di thread di A. Limita il numero di chiamate concorrenti verso ogni dipendenza.

Idempotenza + Retry: le operazioni che possono essere ritentate devono essere idempotenti. Altrimenti un retry su un pagamento = doppio addebito.

Dead Letter Queue: i messaggi che non riescono a essere processati vanno in una coda di errore per analisi e reprocessing manuale. Non perderli.

Perché conta

Il sistema più resistente non è quello che non fallisce mai: è quello il cui fallimento è predicibile, contenuto, e recuperabile. Progettare per il fallimento non è pessimismo: è ingegneria seria.

Esempio

Il Chaos Engineering di Vortex (immaginario)

Vortex è un marketplace con 15 microservizi. Per validare la resilienza, hai proposto un session di Chaos Engineering: spegnete il servizio "notification" (bassa criticità) mentre il sistema è sotto carico. Risultato: il servizio "order" smette di rispondere perché aspetta la risposta di notification con timeout di 30 secondi. L'API intera va down. Il problema: il circuit breaker non era configurato, e il timeout di 30s bloccava tutti i thread del order-service. Fix: timeout 500ms su notification, circuit breaker con soglia 5 errori, degraded mode ("notifica non riuscita, ma ordine confermato"). Dopo il fix: spegnere notification non ha nessun impatto sull'order flow.

Alternativa

Per sistemi meno maturi, non serve subito il chaos engineering. Inizia con Failure Mode Analysis: fai una sessione con il team in cui per ogni dipendenza esterna chiedete "cosa succede se questo cade?". Scrivi le risposte, identifica i casi non gestiti, implementa i circuit breaker mancanti. Più semplice del chaos engineering ma già molto utile.

Cosa NON fare

Non aggiungere retry senza backoff esponenziale e jitter. Un retry immediato su un servizio già in difficoltà lo affossa completamente. Il retry deve dare al servizio il tempo di riprendersi.

****Attenzione!**** Il Circuit Breaker deve avere uno stato "half-open" che permette di testare periodicamente se il servizio si è ripreso. Un circuito che resta aperto forever non è un circuit breaker: è un kill switch. Assicurati che il comportamento "half-open → closed" sia implementato e testato.

****Tip da campo**** Ogni chiamata remota nel tuo sistema dovrebbe avere, come regola base: timeout (< 2s per chiamate sincrone nell'user path), retry max 2-3 volte con backoff, circuit breaker. Se usi .NET usa Polly. Se usi Java usa Resilience4j. Non reinventare questi pattern da zero.

Domande di verifica:

- Come spiegheresti la differenza tra Circuit Breaker e Retry a un junior developer?

- Cosa significa "cascata di fallimenti" e come il circuit breaker la previene?
- Quando è giusto fare "degraded mode" invece di ritornare un errore all'utente?

PARTE 7 — Observability: il debugging distribuito

Il concetto

In un sistema distribuito, non puoi attaccare un debugger. L'observability è la tua capacità di capire dall'esterno cosa sta succedendo dentro il sistema.

I tre pilastri (Observability Triad):

1. Log (structured): eventi discreti che descrivono cosa è successo. JSON su stdout, mai su file. Campi obbligatori: timestamp, level, service, trace_id. Mai PII.
2. Metriche: aggregazioni numeriche nel tempo. I 4 Golden Signals (Google SRE book):
 - Latency: quanto tempo prendono le richieste
 - Traffic: quante richieste al secondo
 - Errors: percentuale di richieste che falliscono
 - Saturation: quanto è "pieno" il sistema (CPU, memoria, code)
3. Trace (distributed): percorso di una singola richiesta attraverso tutti i microservizi. OpenTelemetry è lo standard. Ogni span è un'operazione, il trace è la catena di span.

Best practices:

- Correla sempre log e trace con trace_id e span_id
- Alert sui SLO (Service Level Objectives), non su soglie arbitrarie
- Dashboard per ogni servizio con i 4 Golden Signals
- Runbook per ogni alert: chi chiamare, cosa fare, come verificare il fix

Perché conta

Senza observability, quando il sistema ha un problema alle 3 di notte, sei cieco. Con observability, hai un dashboard che ti dice "il servizio payment ha il 3% di error rate, latency media 2.3s, e il spike è iniziato 12 minuti fa quando è stato deployato questo commit".

Esempio

Come trovi un bug invisibile con il tracing

Un utente segnala che l'acquisto a volte impiega 8 secondi. A volte 200ms. Nessun errore visibile. Senza tracing: impossibile capire. Con tracing: cerchi le trace con latency > 3s per l'endpoint /checkout. Trovi che il 95% ha P99 < 500ms, ma il 5% ha un span anomalo: il servizio "tax-calculator" impiega 7.8s. Zoom sullo span: sta chiamando un servizio SOAP legacy che a volte è lento. Soluzione: timeout aggressivo + cache dei risultati del tax-calculator (i valori cambiano raramente). Problema risolto senza fare rollback, senza debug in produzione, senza downtime.

Alternativa

Se il cliente non ha budget per un full observability stack (Grafana + Prometheus + Jaeger/Tempo), inizia con il minimo vitale:

- Sentry per error tracking (gratis per volumi bassi)
- UptimeRobot per uptime monitoring (gratis)

- Log su CloudWatch o Datadog (costo proporzionale al volume)
- OpenTelemetry SDK nel codice (gratis, poi scegli il backend)

Cosa NON fare

Non loggare tutto a DEBUG in produzione. Non usare log come file su disco senza rotation. Non fare alert su "CPU > 80%" senza contesto: è normale durante un on-sale. E mai loggare dati personali: GDPR violation + security issue.

****Attenzione!**** *L'observability è inutile se nessuno la guarda. Ogni team deve avere un SLO definito ("99.9% di richieste in < 500ms") e un alert che scatta quando viene violato. Senza SLO, i dashboard sono decorazione.*

****Tip da campo**** *Il migliore investimento di observability per una start-up è OpenTelemetry SDK + un singolo trace backend (Grafana Tempo è gratis e potente). Aggiungi il tracing dal giorno 1: retrofittarlo su un sistema esistente è molto più costoso.*

Domande di verifica:

- Come spieghi la differenza tra metriche e trace a un CEO che vuole capire "cosa sta succedendo"?
- Cosa sono i 4 Golden Signals e come li mapperesti su un servizio di ticketing?
- Come costruiresti un SLO per un endpoint di acquisto biglietti?

PARTE 8 — CI/CD e la cultura del deployment frequente

Il concetto

Un deployment è un evento tecnico noioso. Non dovrebbe far paura. Se fa paura, il problema è il processo, non il deployment in sé.

La pipeline CI/CD ideale:

Commit → Code Review → CI (build + test + lint + security scan) → Deploy Staging → Smoke Test → Deploy Production

Tempo ideale: < 20 minuti dall'inizio alla fine per progetti medi.

Principi:

1. Test automatici come gate: nessun deploy se i test falliscono. Nessuna eccezione.
2. Feature flags: disaccoppia deploy da release. Il codice va in produzione ma la feature è "off". La attivi senza deploy.
3. Blue/Green deployment: due ambienti identici. Il traffico viene spostato da "blue" (vecchio) a "green" (nuovo) in modo atomico. Rollback = reindirizza traffico al blue.
4. Canary deployment: il nuovo codice serve un 5% del traffico. Monitora metriche. Se ok, aumenta. Se no, rollback immediato.
5. Database migration forward-only: mai migration che fa DROP TABLE o rimuove colonne usate dalla versione corrente. Il codice deve poter girare sia sulla versione old che new dello schema durante il deployment.

Perché conta

La frequenza di deployment è correlata positivamente con la qualità del software (DORA metrics). Teams che fanno deploy spesso hanno meno bug, recuperano più velocemente dagli incidenti, e sono più veloci nello sviluppare nuove

feature. Non è controintuitivo: è perché i batch piccoli sono più facili da testare e da rollbackare.

Esempio

La trasformazione di DevFlow (immaginario)

DevFlow deployava ogni 2 settimane perché "i deployment sono rischiosi". Con te, in 3 mesi: pipeline CI/CD automatizzata (prima: manuale), test coverage dal 12% al 65%, feature flags implementati sui componenti critici, blue/green deployment configurato. Risultato: sono passati a deployment giornaliero. Il numero di bug in produzione è calato del 40% (non aumentato, come temevano). "Deploy every day" ha costretto il team a scrivere test per ogni feature, a mantenere la pipeline verde, e a fare PR più piccole.

Alternativa

Se il team non è pronto per daily deployment, il target intermedio è "deployment on demand" (quando vuoi, non secondo un calendario fisso). Questo già riduce il batch size e abbassa il rischio per deployment.

Cosa NON fare

Non "congelare i deployment" prima di eventi importanti. Se sei in grado di congelare i deployment, sei in grado di fare rollback. Se non sei in grado di fare rollback, hai un problema più profondo che il congelamento non risolve.

****Attenzione!**** La zero-downtime deployment richiede che il codice e il database migration siano compatibili con la versione precedente durante il deployment (rolling update). Se un nuovo campo diventa NOT NULL durante il rollout, le istanze vecchie che ancora girano andranno in errore. Pianifica le migration in più step.

****Tip da campo**** Il modo più veloce per migliorare la cultura del deployment è ridurre la dimensione dei PR. Un PR con 500 righe di differenza è terrificante. Un PR con 50 righe è gestibile. La dimensione del PR guida automaticamente la frequenza del deploy.

Domande di verifica:

- Qual è la differenza tra Blue/Green e Canary deployment? Quando sceglieresti l'uno e quando l'altro?
- Come implementeresti una database migration zero-downtime su una colonna che deve diventare NOT NULL?
- Come convinceresti un team che "i deployment frequenti sono pericolosi" a cambiare approccio?

PARTE 9 — Security Architecture: sicurezza come proprietà, non aggiunta

Il concetto

La sicurezza non si aggiunge alla fine: si progetta dall'inizio. Security by Design significa che ogni decisione architetturale include una valutazione della superficie di attacco.

Principi fondamentali:

1. Least Privilege: ogni componente accede solo alle risorse che gli servono. Il servizio "billing" non deve poter leggere i dati medici. Il worker asincrono non deve avere accesso admin al database.
2. Defense in Depth: ogni livello ha la sua difesa. Se un attaccante bypassa il WAF, trova ancora l'autenticazione. Se bypassa l'autenticazione, trova l'autorizzazione a livello di dato.
3. Zero Trust: non fidarti della rete interna. Ogni chiamata, anche interna, deve essere autenticata e autorizzata. mTLS tra microservizi, JWT validation su ogni hop.

4. Input validation: valida tutto all'ingresso. Mai fidarsi dei dati che arrivano dall'esterno. OWASP Top 10 come checklist minima.

5. Secret management: mai secret in codice, file di configurazione non crittografati, o variabili d'ambiente chiare. HashiCorp Vault, AWS Secrets Manager, Azure Key Vault.

6. Dependency scanning: le librerie di terze parti sono la tua superficie di attacco più grande. Snyk, OWASP Dependency-Check, Dependabot: integra in CI, non come after-thought.

Perché conta

Una violazione di sicurezza in un sistema che gestisce dati di pagamento, dati personali, o dati di biglietti sportivi non è un bug: è un incidente con conseguenze legali, reputazionali, e finanziarie gravissime. Il costo di un breach è sempre enormemente superiore al costo di prevenirlo.

Esempio

Il JWT senza validazione dell'audience

Un sistema di ticketing usava JWT per l'autenticazione. Lo stesso JWT era valido sia per l'app B2C che per l'API B2B. Un attaccante con accesso a un account B2C poteva usare il token sulle API B2B (che non validavano il campo aud). Risultato: accesso non autorizzato ai dati di tutti gli ordini. Fix: aggiungere validazione aud + iss su ogni API. Tempo di scoperta: 8 mesi. Tempo di fix: 30 minuti. Danno: significativo.

La lezione: la validazione del JWT non si limita alla firma. Tutti i claim (iss, aud, exp, nbf, sub) devono essere validati e interpretati correttamente.

Alternativa

Per sistemi piccoli che non possono permettersi un security architect dedicato, una Security Review Checklist semplificata è il minimo accettabile:

- [] Nessun secret in repository git
- [] HTTPS ovunque, anche internamente
- [] JWT validazione completa (firma + claims)
- [] Input sanitization su tutti i form e API
- [] Dependency scanning in CI
- [] Logging di eventi di sicurezza (login failed, access denied)
- [] Rate limiting sulle API pubbliche

Cosa NON fare

Non mettere la sicurezza in queue come "lo facciamo dopo". Non fare "security by obscurity" (nascondere l'API key nella logica del frontend). Non riusare secret tra ambienti (dev secret = prod secret è una vulnerabilità).

****Attenzione!**** Il maggior rischio non è l'attaccante sofisticato: è il developer che per pigrizia mette la chiave API in chiaro nel file di configurazione committato. I secret leak su git sono la causa numero 1 di breach nelle aziende tech. Usa `git-secrets` o `trufflehog` come pre-commit hook.

****Tip da campo**** Fai un threat modeling con il team: "se fossi un attaccante, come attaccherei questo sistema?" Usa STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, DoS, Elevation of privilege) come checklist. Una sessione di 2 ore con il team scopre vulnerabilità che nessun tool automatico trova.

Domande di verifica:

- Qual è la differenza tra autenticazione e autorizzazione? Come si implementano in un sistema a microservizi?

- Come si fa il "rotation" di un secret che è già in produzione senza downtime?
- Quali sono i 3 rischi principali del JWT che ogni developer deve conoscere?

PARTE 10 — Il Tuo Prodotto: dall'Assessment al Retainer

Il concetto

Come si struttura il track ARCH nel Modello Gamma:

Prodotto | Prezzo | Durata | Output

Architecture Conversation | 500–800€ | 3 ore | Documento priorità (2 pag.)

Architecture Assessment | 2.000–5.000€ | 2-3 sett. | Full report + roadmap

Architecture Sprint | 3.000–8.000€ | 4-6 sett. | Design + prototipo + decision record

Architecture Retainer | 1.500–3.000€/mese | mensile | Governance + review + decision support

Team Training | 1.000–3.000€ | 1-2 gg | Workshop su pattern specifico

Target clienti:

- Start-up Series A-B con team da 10-30 dev che iniziano a sentire le prime frizioni
- Scale-up con problemi di velocità di deployment
- Aziende che stanno acquisendo o venendo acquisite (due diligence tecnica)
- Aziende che vogliono portare il team da "junior" ad "architetti"

Le obiezioni ARCH

"Ho già un CTO, non ho bisogno di un consulente"

"Il tuo CTO è nel sistema ogni giorno. Io porto uno sguardo esterno. Il medico non opera se stesso. L'architettura si beneficia molto di una review indipendente proprio perché chi è dentro non vede più le assunzioni implicite."

"Non ho budget per refactoring"

"Capisco. Il mio approccio è identificare quick win: i 3 cambiamenti che portano il 70% del beneficio con il 30% dell'effort. Partiamo da lì, senza commitment su un refactoring totale."

"L'architettura la gestiamo internamente"

"Ottimo, vuol dire che avete persone capaci. Quello che porto io è un secondo opinion su decisioni che avranno impatto a lungo termine. Non per fare il vostro lavoro: per renderlo più solido."

Upsell naturali

- Da Assessment → Architecture Sprint (hai trovato i problemi, ora risolviamoli insieme)
- Da Sprint → Retainer (hai visto il valore, vuoi continuità?)
- Da ARCH → LEAD (il problema tecnico è anche un problema di leadership del team)
- Da ARCH → FCTO (l'architettura è solo parte di un bisogno più ampio di CTO fractional)

****Tip da campo**** L'Architecture Assessment per una due diligence M&A è uno dei prodotti più remunerativi: le aziende acquisite valgono molto meno se il tech è un disastro. Una startup che sta per essere acquisita per €5M pagherà volentieri €5K per una tech due diligence che protegge (o migliora) il deal.

Domande di verifica:

- Come differenzieresti il tuo servizio ARCH rispetto a un'altra consulente IT generico?
- Quale è il momento migliore per proporre un Architecture Retainer?
- Come gestisci il caso in cui il tuo assessment rivela che il CTO interno ha fatto scelte architetturali molto sbagliate?

PARTE 11-14 — Pattern Avanzati, Team Topology, FinOps, e Piano di Lancio

(Sezioni sintetiche per completezza — ogni tema merita un workshop dedicato)

PARTE 11 — Team Topology e Cognitive Load

I 4 tipi di team (Conway's Law applicato):

- Stream-aligned: segue il flusso di valore end-to-end (es. team "checkout")
- Platform: abilita gli altri team (infrastruttura, CI/CD, shared services)
- Enabling: consulenza e formazione agli stream-aligned (come il tuo ruolo)
- Complicated-subsystem: expertise profonda su un componente complesso (es. algoritmo di pricing)

Il principio: il design dei team determina il design dell'architettura (Conway's Law). Per cambiare l'architettura, cambia prima i team.

PARTE 12 — FinOps: l'architettura che non manda in bancarotta

I costi cloud sono spesso il 2° o 3° costo operativo dopo le persone. Principi:

- Right-sizing: non usare un'istanza da 64GB per un job che ne usa 4
- Spot/preemptible: workload batch e non-critici su istanze spot = -70% costo
- Reserved instances: per workload stabili e prevedibili, commitment 1 anno = -40%
- Cost allocation tags: ogni risorsa cloud deve avere tag team/progetto/ambiente
- Waste detection: trova le risorse non utilizzate ogni mese (snapshot orfani, load balancer senza backend, database fermi)

PARTE 13 — Platform Engineering: costruire la "golden path"

Il Platform Engineering è la pratica di costruire un'Internal Developer Platform (IDP): l'insieme di strumenti, template, e automazioni che rendono gli sviluppatori produttivi senza doversi preoccupare dell'infrastruttura.

Componenti tipici di un IDP:

- Self-service provisioning (crea un nuovo microservizio in 5 minuti)
- Golden path templates (Cookiecutter, Backstage)
- Osservabilità pre-configurata (logging, metriche, tracing già attivi)
- CI/CD standard (non ogni team reinventa la pipeline)
- Secret management integrato

Il tuo ruolo come consulente ARCH: aiutare il cliente a capire se è pronto per il Platform Engineering, e progettare la "golden path" minima che porta il maggior valore.

PARTE 14 — Il tuo Piano di Lancio ARCH

Mese 1: Definisci il tuo focus (start-up? scale-up? settore verticale?). Crea l'offer card per Architecture Assessment. Identifica 10 aziende target. Fai 3 Architecture Conversation gratuite per validare il posizionamento.

Mese 2: Primo Assessment pagato. Case study (anonimo). LinkedIn post sui pattern che hai trovato. Network con CTO italiani (meetup, community).

Mese 3: Secondo Assessment. Proponi primo Architecture Sprint. Valuta primo retainer. Costruisci il tuo template di Assessment per efficienza.

CHECKLIST — Architecture Consultant

Pre-engagement

- ☐ Ho capito la dimensione del team e del sistema
- ☐ Ho identificato il tipo di problema (tecnico / architetturale / organizzativo)
- ☐ Ho una stima del impatto business del problema
- ☐ Ho identificato gli stakeholder chiave (CTO, tech lead, team)

Assessment

- ☐ Ho analizzato la struttura del codice (boundary, accoppiamento)
- ☐ Ho fatto git hotspot analysis
- ☐ Ho misurato la frequenza di deployment attuale
- ☐ Ho valutato l'osservabilità esistente
- ☐ Ho intervistato almeno 3 persone del team tecnico
- ☐ Ho prodotto l'Architecture Map (C4 level 1-2)
- ☐ Ho prodotto il Risk Register
- ☐ Ho prodotto la Roadmap con quick wins e interventi strategici

Delivery

- ☐ Ho consegnato il report in un formato fruibile (non un paper accademico)
- ☐ Ho presentato i risultati al team (non solo al CTO)
- ☐ Ho spiegato il "perché" di ogni raccomandazione, non solo il "cosa"
- ☐ Ho definito le metriche di successo per la roadmap
- ☐ Ho proposto il next step naturale

10 SCENARI PRATICI CON PERSONAGGI INVENTATI

Scenario 1 — La Start-up che cresce troppo veloce

Paolo Righi, CTO di Fluxx (SaaS HR, 12 sviluppatori, da 0 a 80K utenti in 18 mesi)

Paolo ha costruito tutto da solo nella prima fase. Ora ha 12 developer e il sistema è un Rails monolite. Deploy ogni 3 settimane, 4-5 bug critici/mese. La tua analisi: 3 hotspot (billing, notifications, integrations), 0 circuit breaker, 15% test coverage. Proposta: 6 mesi di lavoro, prima il modulo billing come "bounded context" nel monolite (non microservizio), poi automazione CI/CD, poi test coverage a 60%. Paolo accetta. 6 mesi dopo: deploy ogni giorno, bug critici < 1/mese, il team è di nuovo entusiasta.

Scenario 2 — Il Distributed Monolith

Marco Coletti, VP Engineering di Prism (marketplace B2B, 8 microservizi, 25 sviluppatori)

Marco ha 8 microservizi ma ogni deploy richiede di aggiornare 6 dei suoi 8 in sequenza. Il "microservizio A" fa query dirette al database del "microservizio B". Diagnosi: distributed monolith. Non serve aggiungere microservizi: serve eliminare le dipendenze nascoste. Piano: 4 mesi per identificare e rimuovere tutti i coupling impliciti, introdurre eventi (Kafka) come canale di comunicazione, e rendere ogni servizio davvero indipendente. Risultato: ogni team può deployare in autonomia.

Scenario 3 — La Tech Due Diligence

Sofia Vinci, CEO di TechSnap (start-up acquisita per 3M€)

Il buyer ha richiesto una tech due diligence. Sofia ti ingaggia. Hai 5 giorni. Trovi: buona codebase su parti core, ma dipendenza critica da una libreria abbandonata per il modulo più importante (cron jobs), e un database con 0 backup automatici in produzione. Report onesto: rischio medio-alto, con piano di remediation chiaro. Il buyer riduce il valuation del 10% invece di cancellare il deal. Sofia non è felicissima, ma è infinitamente meglio di un deal che salta o di scoprire i

problemi post-acquisizione.

Scenario 4 — Il CTO che vuole imparare

Elena Mori, 32 anni, CTO di una start-up proptech da 6 sviluppatori

Elena è tecnicamente brillante ma ha sempre costruito da sola. Vuole capire "come si fa architettura in modo professionale". Non vuole che tu faccia le sue scelte: vuole imparare a farle bene. Proposta: Architecture Mentoring per 6 mesi, un call da 90 min/settimana + review delle sue decisioni + lettura guidata di libri chiave (Clean Architecture, Software Architecture: The Hard Parts). Elena dopo 6 mesi prende decisioni architetturali molto più solide, con trade-off espliciti e ADR documentati. Diventa una tua ambassador.

Scenario 5 — Il Legacy che nessuno vuole toccare

Filippo Amato, Head of Engineering di RetailChain (600K righe PHP, 2006)

Il sistema legacy PHP ha 600K righe, zero test, e gestisce 2M€/giorno di transazioni. Nessuno vuole toccarlo. Il tuo approccio: non riscrivere tutto. Usa lo Strangler Fig: identifica 3 nuove funzionalità che possono essere costruite come microservizi separati, con il legacy come "fallback". Testa. Impara i confini. Gradualmente il traffico va sui nuovi servizi. Il legacy si svuota. In 2 anni, il legacy gestisce il 30% del traffico invece del 100%. Zero downtime, zero grande riscrittura.

Scenario 6 — Il Database del Terrore

Carla Fontana, Backend Lead di QuickPay (fintech, 5M transazioni/giorno)

Il database principale di QuickPay ha 800 GB, 340 tabelle, e un DBA che "conosce tutto" ma non ha documentato nulla. Il DBA sta per andare in pensione. Carla è terrorizzata. Tuo piano: documentation sprint (2 settimane per mappare le tabelle critiche, le relazioni, e i query pattern principali), knowledge transfer strutturato con il DBA, introduzione di read replicas per le query analitiche, e piano di archivio dati storici (il 40% del DB è dati > 3 anni mai più acceduti).

Scenario 7 — Il Multi-Tenant Gone Wrong

Roberto Bianchi, CTO di CloudForm (SaaS B2B, 200 clienti aziendali)

CloudForm ha costruito il multi-tenancy sbagliato: tutti i clienti sullo stesso database, con un campo tenant_id su ogni tabella. Ora un bug ha esposto dati di un tenant a un altro (data leak). Urgenza massima. Piano immediato: audit completo di ogni query per verificare che filtrino sempre per tenant_id, introduzione di row-level security a livello database, test automatici che verificano l'isolamento. Piano a medio termine: migrazione verso schema separato per tenant (più sicuro, più isolato). Dopo 3 mesi: zero data leak, audit trail completo, e certificazione SOC2 possibile.

Scenario 8 — Il Monolite Che Scala

Tommaso Galli, CEO di EventStream (ticketing, 100K eventi/anno)

Tommaso pensa di dover "fare microservizi" perché "il monolite non scala". Assessment: il monolite scala benissimo — reggono 50K req/s con il setup attuale. Il problema reale: il deployment richiede 2 ore di testing manuale. La soluzione non è l'architettura: sono i test automatici e la pipeline CI/CD. Automazione CI/CD in 6 settimane: deployment time da 2 ore a 20 minuti. Nessun microservizio. Nessuna riscrittura. Problema reale risolto.

Scenario 9 — Il Team che Non Si Allinea

Nadia Conti, VP Eng di DataHub (analytics SaaS, 4 team da 6 sviluppatori)

I 4 team di Nadia fanno scelte tecnologiche diverse: chi usa Kafka, chi RabbitMQ, chi HTTP sync. Chi usa Postgres, chi MongoDB, chi DynamoDB. Le integrazioni tra team sono un inferno. Tuo lavoro: architecture governance — non un "diktat" tecnologico ma una serie di ADR condivise, un Tech Radar interno, e un processo di "RFC" per proposte architetturali significative. Dopo 6 mesi: le nuove scelte tecnologiche sono coerenti, le integrazioni nuove sono più semplici, e il team senior sente di avere voce in capitolo.

Scenario 10 — Il Cliente Ideale (esiste!)

Andrea Russo, CTO di Pulse (healthcare SaaS, 18 sviluppatori)

Andrea ha già fatto il lavoro di base: test coverage 75%, CI/CD funzionante, code review process. Il suo problema è strategico: "so che nei prossimi 12 mesi andremo in 5 nuovi mercati europei e la nostra architettura non regge l'internazionalizzazione". Perfetto: non stai risolvendo un disastro, stai progettando per la crescita. Architecture Sprint di 6 settimane: design multi-tenancy con isolamento per mercato, internazionalizzazione del data model, event-driven per il backoffice internazionale. Il lavoro è pulito, il team è bravo, e il risultato sarà duraturo.

GLOSSARIO ARCH

ADR (Architecture Decision Record) — Documento che registra una decisione architetturale: il contesto, le opzioni valutate, la scelta fatta, e il razionale. Ogni decisione significativa deve avere il suo ADR.

Bounded Context — In DDD, un confine esplicito entro cui un modello di dominio è coerente. Ogni bounded context ha il suo linguaggio, i suoi dati, e le sue regole.

Circuit Breaker — Pattern che interrompe le chiamate verso un servizio degradato dopo N fallimenti, permettendo al sistema di recuperare.

CQRS — Command Query Responsibility Segregation. Separazione del modello di scrittura da quello di lettura.

Cognitive Load — La quantità di "complessità mentale" che un team deve gestire. Troppo cognitive load → errori, lentezza, burnout.

Conway's Law — Le organizzazioni producono sistemi che rispecchiano la loro struttura di comunicazione. Vuoi cambiare l'architettura? Cambia la struttura del team.

Event Sourcing — Pattern in cui lo stato è derivato da una sequenza immutabile di eventi, invece di essere salvato direttamente.

Fitness Function — Un test automatizzato che verifica che l'architettura rispetti una proprietà definita (es. "nessun modulo dipende direttamente dal database degli altri").

Golden Signals — Le 4 metriche fondamentali di un servizio: latency, traffic, errors, saturation.

Idempotenza — Proprietà di un'operazione che può essere eseguita N volte con lo stesso effetto di eseguirla 1 volta.

Platform Engineering — La pratica di costruire un'Internal Developer Platform che abilita gli sviluppatori a fare deployment, osservabilità, e provisioning in self-service.

SLO (Service Level Objective) — L'obiettivo di qualità del servizio (es. "99.9% delle richieste in < 500ms").

Strangler Fig Pattern — Tecnica di migrazione che costruisce il nuovo sistema intorno al vecchio, reindirizzando gradualmente il traffico.

Tech Debt — Il costo futuro derivante da scelte tecniche sub-ottimali prese nel passato per velocità o vincoli di budget.

TOKEN & COSTO STIMATO

Voce | Valore

Lunghezza documento | ~1.050 righe, ~10.500 parole

Token input stimati | ~4.000 (contesto + sistema)

Token output stimati | ~12.500

Modello | Claude Opus (Bedrock)

Costo output stimato | ~\$0.65

Costo totale sessione | ~\$0.75

I costi sono stime basate su tariffe pubbliche Anthropic/AWS Bedrock a maggio 2026.